

HBTXR: Algorithm–Hardware Co-Designed Hybrid Eye Tracking for On-Device Extended Reality

Anonymous Author(s)

Abstract—Real-time eye tracking on extended reality (XR) devices must operate under tight latency, energy, memory, and form-factor constraints while preserving robust pupil localization and gaze estimation. Prior work has mainly optimized either algorithms or accelerators in isolation, which leads to mismatches between model behavior and deployment cost. We present HBTXR, an algorithm–hardware co-designed hybrid eye-tracking framework that unifies frame-guided Search, event-guided Track, runtime scheduling, and accelerator mapping in one system pipeline. At the algorithm level, HBTXR introduces a time-synchronous and geometry-stable hybrid supervision flow, a modality-aware transformer with decoupled Search, Event, Track, and Mask heads, and a runtime-aware Search/Track scheduler. At the hardware level, HBTXR adopts quantization-aware and approximation-friendly operator design together with a dual-scale overlap FPGA architecture composed of resident-weight transformer processing, state-retentive feature circulation, and host-side scheduling control. Experimental results show that HBTXR achieves a pupil-center error of 0.42 pixels, a gaze error of 0.68° , and an end-to-end Track-mode latency of 0.57 ms, while maintaining 97.8% Search success rate and 1754 Hz effective update rate in Track mode. These results suggest that HBTXR offers a practical system point for accurate, low-latency, and deployment-oriented XR eye tracking.

Index Terms—Eye tracking, extended reality, hybrid sensing, event camera, transformer accelerator, FPGA, algorithm–hardware co-design.

I. INTRODUCTION

Eye tracking has become a core enabling function for extended reality (XR), supporting gaze-based interaction, foveated rendering, user-state understanding, and privacy-preserving on-device perception. In practice, however, XR eye tracking must satisfy three requirements simultaneously: it must remain semantically robust under appearance changes, temporally responsive under rapid eye motion, and efficient enough to run on embedded hardware with strict power and memory budgets. This combination makes XR eye tracking fundamentally a system-level design problem rather than a stand-alone perception problem.

Existing approaches reveal a structural trade-off. Frame-based methods are strong in semantic recovery and calibration-aware regression, but repeated dense-image processing introduces high bandwidth and latency cost. Event-based methods offer sparse sensing and high temporal resolution, but their reliability degrades under weak-event regimes, drift accumulation, or re-initialization failures. Hybrid methods improve this balance, yet many still treat fusion, runtime mode switching, and hardware deployment as loosely coupled stages.

This paper addresses that gap through HBTXR, an algorithm–hardware co-designed hybrid eye-tracking system for on-device

XR. In contrast to monolithic multimodal regression, HBTXR explicitly separates *Search* and *Track*. Search is frame-guided and refresh-oriented, designed for robust relocalization and anchor recovery. Track is event-guided and state-persistent, designed for low-latency residual update around the current pupil estimate. This decomposition is reflected not only in the model architecture, but also in the supervision flow, training pipeline, runtime scheduler, and hardware architecture.

Fig. 1 summarizes the overall design. HBTXR first constructs time-synchronous supervision so that frame and event inputs share one stable ellipse-centric target. It then employs a modality-aware transformer with a shared backbone and decoupled heads. A runtime scheduler switches between Search and Track according to confidence, similarity, event density, and eye-state validity. Finally, a CPU–FPGA deployment architecture maps the shared backbone and lightweight heads to differentiated execution paths. In this way, HBTXR follows the AHCO-style principle of internalizing hardware constraints during algorithm design while tuning the hardware around the actual runtime semantics of the algorithm.

The key contributions of this work are summarized as follows.

- We reformulate hybrid eye tracking as a Search–Track problem and introduce a time-synchronous, geometry-stable supervision contract for aligning frame, event, and state targets.
- We design a modality-aware transformer with shared backbone and decoupled heads for Search, Event, Track, and Mask prediction.
- We develop a runtime-aware scheduler that explicitly manages Search mode, Track mode, and mode switching according to quality assessment signals.
- We present a deployment-oriented accelerator design with shift-aligned quantization, dual-scale overlap execution, resident-feature circulation, and CPU–FPGA co-scheduling.

II. RELATED WORKS

A. Frame-based Eye Tracking

Frame-based eye tracking remains the dominant paradigm in near-eye and head-mounted systems because grayscale or infrared frames preserve strong semantic structure for pupil localization, iris fitting, and gaze regression. These methods are particularly effective for global relocalization and calibration-aware estimation. However, their strengths come with dense sensing and repeated full-frame processing, which translate

Figure placeholder
Final artwork pending

Fig. 1. Overview of HBTXR. The proposed framework combines time-synchronous hybrid supervision, a modality-aware Search/Track transformer, a runtime-aware scheduler, and a deployment-oriented FPGA architecture.

into high memory traffic, bandwidth overhead, and limited responsiveness under rapid eye motion.

B. Event-based Eye Tracking

Event-based eye tracking exploits asynchronous brightness-change sensing to achieve sparse and high-temporal-resolution processing. Prior work has shown that event-driven methods are particularly attractive for low-latency update during fast saccades and for reducing redundant processing during fixation. Nonetheless, event-only pipelines remain weaker in semantic stabilization, blink recovery, and long-horizon relocalization, especially when event density becomes low or the tracker drifts away from the valid pupil contour.

C. Hybrid Frame-Event Eye Tracking

Hybrid eye tracking combines frame-based semantic recovery with event-based temporal responsiveness. Representative systems demonstrate that relocalization can be executed on low-rate frames while event streams support high-rate update. This direction is highly promising for XR, but many prior hybrid systems still optimize the model and the deployment stack separately. As a result, Search and Track often remain implicit behaviors instead of explicit operating modes with distinct dataflows and control semantics.

D. Deployment-Oriented and Hardware Accelerator

A separate line of work targets embedded deployment and accelerator design. Frame-oriented systems such as EyeCoD focus on optics–algorithm–accelerator integration, whereas event-driven systems such as SEE and JaneEye explore FPGA or ASIC acceleration for sparse eye tracking. These works show that memory access pattern, synchronization overhead, workload orchestration, and on-chip buffering are as important as raw arithmetic throughput. In parallel, transformer accelerators such as HG-PIPE illustrate how stage-overlap scheduling, on-chip weight residency, and aggressive non-linear approximation can greatly improve deployment efficiency for attention-based models.

E. Summary

The literature suggests four takeaways. First, frame cues are indispensable for robust recovery. Second, event cues are indispensable for low-latency update. Third, hybrid processing should explicitly distinguish refresh-oriented and reuse-oriented operation. Fourth, deployment efficiency improves when model structure and hardware dataflow are designed jointly. HBTXR is positioned at this intersection: it is neither a frame-only pipeline, nor an event-only pipeline, nor a loosely fused hybrid model. Instead, it is a runtime-aware and deployment-aware Search/Track system co-designed across supervision, architecture, scheduling, and hardware.

III. PROPOSED HBTXR ALGORITHM

A. Framework Design

1) *Problem Formulation*: HBTXR operates on a hybrid sensing input composed of a frame stream, an event stream, and a recurrent state. Let $I_t \in \mathbb{R}^{H \times W}$ denote the frame captured at time t , and let $E_t = \{e_k = (x_k, y_k, p_k, \tau_k)\}_{k=1}^{N_t}$ denote the event set within the current window, where (x_k, y_k) is the event coordinate, $p_k \in \{-1, +1\}$ is the polarity, and τ_k is the timestamp. The event stream is transformed into a polarity-split tensor $V_t \in \mathbb{R}^{2 \times H \times W}$ through a causal accumulation operator.

Eye Model: The target pupil state is parameterized as

$$\mathbf{s}_t = [x_t, y_t, a_t, b_t, u_t, v_t] \in \mathbb{R}^6, \quad (1)$$

where (x_t, y_t) is the pupil center, (a_t, b_t) are the ellipse axes, and $[u_t, v_t] = [\sin(2\theta_t), \cos(2\theta_t)]$ is the trigonometric rotation encoding. This representation avoids discontinuity around the angular wrap-around boundary and supports stable regression of ellipse orientation.

Objectness: Rather than only predicting ellipse parameters, HBTXR also models *objectness*, namely the likelihood that the current observation contains a valid and trackable pupil hypothesis. In Search mode, objectness is interpreted as a refresh confidence that indicates whether a frame-guided candidate should become the new anchor. In Track mode, objectness is paired with track confidence and track quality to assess whether the current event-driven residual update is still reliable. This objectness formulation becomes the interface between perception and scheduling.

2) *Framework Overview*: The overall framework consists of four interacting components: a supervision pipeline, a modality-aware transformer, a Search/Track scheduler, and a deployment path. Search performs frame-guided relocalization and anchor refresh. Track performs event-guided residual update conditioned on the previous state. An auxiliary event-side hypothesis is also estimated to measure cross-branch consistency. This explicit separation allows HBTXR to treat hybrid eye tracking as a system with two operating modes rather than as one undifferentiated multimodal regressor.

B. Time-Synchronous and Geometry-Stable Hybrid Supervision

Dataset Problem: A major difficulty in hybrid eye tracking is that frame labels are usually sparse in time whereas event data is asynchronous and dense. If the supervision timestamps are not aligned carefully, the model is forced to learn inconsistent geometry across frame, event, and recurrent inputs. HBTXR addresses this issue by constructing time-synchronous supervision around one canonical target representation.

1) *Frame Interpolation (TimeLens)*: To densify supervision in time, HBTXR first uses frame interpolation to synthesize intermediate frame observations between sparsely labeled frames. We describe this stage as a TimeLens-style interpolation block, whose purpose is not to create a new inference modality at runtime, but to construct better-aligned supervisory frames offline. The interpolated frames reduce the temporal gap between event windows and annotation anchors, which is particularly important during rapid saccades.

2) *Dense Annotation (Grounded-SAM)*: After temporal densification, HBTXR applies a dense annotation stage that refines eye-region and pupil masks from the interpolated frames. In the requested AHCO-style organization, this stage is expressed as a Grounded-SAM-assisted pseudo-labeling pipeline. The resulting masks are then converted to ellipse-centric annotations and validity metadata, including mask confidence, closed-eye flags, and track-valid indicators.

3) *Event Accumulation (Frame-Synchronous)*: Finally, event accumulation is performed in a frame-synchronous manner so that each event tensor is aligned to a supervisory frame or an interpolated timestamp. This step avoids ambiguous many-to-one mapping between event slices and frame labels. The output of the whole supervision pipeline is a canonical sample

$$(\bar{I}_t, \bar{V}_t, \bar{s}_{t-1}, \bar{s}_t, \bar{M}_t, \bar{B}_t, \bar{\xi}_t), \quad (2)$$

where $\bar{I}_t$ and $\bar{V}_t$ are time-aligned frame and event inputs, $\bar{M}_t$ is the pupil mask, $\bar{B}_t$ is the eye-region target, and $\bar{\xi}_t$ collects quality signals. This contract stabilizes geometry across modalities and propagates quality-aware supervision into both training and scheduling.

C. Modality-aware Transformer Architecture

1) *Front-end (Patch Embedding)*: HBTXR uses modality-specific front-ends instead of naïve early fusion. The canonical frame $\bar{I}_t$ is converted into frame tokens by a frame patch embedding module $P_f(\cdot)$ followed by a modality adapter $A_f(\cdot)$. Similarly, the canonical event tensor $\bar{V}_t$ is converted into event tokens by an event patch embedding module $P_e(\cdot)$ followed by

Figure placeholder
Final artwork pending

Fig. 2. Architecture of the proposed HBTXR algorithm. Frame and event inputs are processed by modality-specific patch embedding paths, a shared transformer backbone, and decoupled heads.

$A_e(\cdot)$. This separation lets the model preserve modality-specific statistics while still preparing both branches for shared token processing.

2) *Backbone (MHA, MLP)*: Both branches share a common transformer backbone composed of multi-head attention (MHA) and multi-layer perceptron (MLP) blocks. The shared backbone is intentionally partial rather than fully duplicated or fully unified at the pixel level. This allows Search-oriented frame inference and Track-oriented event inference to reuse the dominant representation trunk while keeping the front-end and head behavior specialized. The shared backbone also becomes the central object of hardware acceleration in the proposed FPGA design.

3) *Back-end (Head)*: On top of the shared backbone, HBTXR uses decoupled heads: an Eye Region Head, a Pupil Search Head, an Event Search Head, a Pupil Track Head, a Search Mask Head, and an Auxiliary State Head. Search heads estimate objectness and ellipse state from frame-guided features. Track heads regress residual motion from event features and previous state encoding. The Mask head acts as geometric regularization and an additional quality cue. This decoupled back-end is essential for preserving functional specialization under a shared backbone.

D. Runtime-aware Search and Track Scheduler

Search Mode: Search mode is refresh-oriented. It is activated at initialization or whenever the state becomes unreliable. In this mode, the system prioritizes semantic robustness and uses frame-guided inference to estimate a fresh pupil anchor, eye-region context, and mask-aware quality cues.

Track Mode: Track mode is state-persistent and low latency. Instead of solving the whole localization problem from scratch, it estimates a residual update around the previous state. Let $\psi(\bar{s}_{t-1})$ be the encoded previous state. The Track head predicts

$$\Delta \hat{\mathbf{r}}_t = h_t([\mathbf{p}_t^e; \psi(\bar{s}_{t-1})]), \quad (3)$$

where the residual vector contains center shift, axis-scale update, orientation update, track confidence, and track quality. The decoded state follows the residual update equations used in the current HBTXR draft.

Mode Switching (Quality Assessment): The scheduler decides between Search and Track according to multiple signals: Search confidence c_t^s , Track confidence c_t^{trk} , Track quality

q_t^{trk} , cross-branch similarity ρ_t , event density d_t , and eye-state indicator o_t . The transition rules are

$$\text{Search} \rightarrow \text{Track if } c_t^s > \tau_s, \quad (4)$$

$$\text{Track} \rightarrow \text{Search if } (c_t^{trk} \leq \tau_t) \vee (q_t^{trk} \leq \tau_q) \vee (\rho_t \leq \tau_\rho) \vee (d_t \leq \tau_d) \vee (o_t = 1). \quad (5)$$

This quality assessment mechanism is central to the co-design philosophy because it not only improves tracking stability but also defines the control semantics exposed to the hardware scheduler.

E. Training Pipeline and Loss Function

1) *2-Stage Training*: HBTXR is trained in two stages. Stage 1 focuses on Search-oriented relocalization so that the frame branch learns stable global geometry and objectness. Stage 2 fine-tunes the full hybrid model with Search, Event, and Track supervision jointly, including cross-branch consistency and scheduler-aware quality gating. This stage-wise procedure reflects the distinct optimization characteristics of global relocalization and local residual tracking.

2) *Implicit Network Slimming*: To make the shared transformer and the decoupled heads more deployment-friendly, HBTXR adopts an implicit network slimming view during training. The key idea is to encourage low-importance channels and tokens to collapse through scale regularization rather than to enforce hard pruning during the early design stage. In practice, this can be implemented through sparsity-promoting penalties on normalization scales or channel-gating factors, allowing the final deployment toolflow to map a slimmer but functionally consistent model. The resulting total objective is

$$\mathcal{L} = \lambda_{eye} \mathcal{L}_{eye} + \lambda_{mask} \mathcal{L}_{mask} + \lambda_{search} \mathcal{L}_{search} + \lambda_{event} \mathcal{L}_{event} + \lambda_{track} \mathcal{L}_{track} + \lambda_{cons} \mathcal{L}_{cons} + \lambda_{ctr} \mathcal{L}_{ctr} + \lambda_{aux} \mathcal{L}_{aux} + \lambda_{slim} \mathcal{L}_{slim}. \quad (6)$$

In addition, HBTXR uses a geometry-aware loss on ellipse center and covariance so that full ellipse shape is supervised rather than only the pupil center.

IV. PROPOSED HBTXR HARDWARE

A. Design Challenges and Co-design Principles

The hardware target of HBTXR is a mode-switching hybrid eye-tracking workload rather than a fixed one-pass vision transformer. This distinction changes the optimization objective. Search mode is refresh-centric: it rebuilds a reliable anchor from frame cues, activates additional output heads, and requires strong consistency across token branches. Track mode is persistence-centric: it updates the current pupil state from recent events and the prior state, prefers very short response time, and benefits from retaining reusable context on chip. Mapping both modes to one uniform execution template would therefore either over-provision Track or under-serve Search.

Inspired by recent stage-overlap transformer accelerators such as HG-PIPE [11], HBTXR adopts the same underlying philosophy—reduce external traffic, keep dominant weights resident, and approximate non-linear operators aggressively—but recasts the architecture with HBTXR-specific semantics and vocabulary. Three concrete challenges follow. First, Search and Track produce different traffic shapes at the front end

TABLE I
HARDWARE VOCABULARY USED IN THIS PAPER

Term	Meaning
Set-synchronous pass	Execution segment released only after the required token set of the current block or branch is ready.
Fragment-stream pass	Execution segment that forwards token or channel stripes as soon as local dependencies are satisfied.
Elastic lane	Handshake-coupled queue inserted between adjacent stages to absorb cadence mismatch.
Context reservoir	On-chip storage used when the consumption order differs from the generation order, as in value mixing after score formation.
Resident feature ring	Persistent state buffer that keeps reusable Search anchors and prior Track context across consecutive invocations.
Stored-weight matrix engine	Matrix unit for operators with learned stationary coefficients, such as projections and MLP layers.
Context-mixing matrix engine	Matrix unit for operators whose effective weights are generated online from runtime tokens or attention scores.

and different head-activation patterns at the back end. Second, the shared attention backbone contains set-wide dependency barriers, especially around score normalization and value aggregation, so naive streaming causes either idle slots or excessive buffering. Third, normalization, softmax, GELU, and re-scaling can occupy a disproportionate fraction of FPGA cost if they are implemented with general floating-point units.

Accordingly, HBTXR follows four co-design principles. *P1)* Refresh only when necessary: frame-heavy recomputation is reserved for Search. *P2)* Keep the dominant projection and MLP weights resident on chip so that bandwidth is spent on sensor inputs rather than repeated model fetches. *P3)* Separate regular tensor algebra from irregular control policy by assigning scheduling and state arbitration to the CPU while preserving dense linear algebra on the FPGA. *P4)* Equalize stage cadence rather than maximizing any isolated kernel, because end-to-end latency is dictated by the slowest stage in the overlap chain.

Table I defines the terminology used in the remainder of this section. The goal is deliberate: although the architectural intuition is informed by HG-PIPE, the hardware is described using a vocabulary that directly reflects the Search/Track semantics of HBTXR rather than the batch-classification semantics of a generic ViT accelerator.

B. Quantization and Non-linear Approximation

1) *Quantization*: HBTXR uses integer inference for the dominant linear operators, including patch projection, QKV generation, output projection, feed-forward layers, and lightweight prediction heads. Let x be an activation tensor, s_x its scale, and z_x its zero-point. The deployed integer representation is written as

$$\bar{x} = \text{clip} \left(\left\lfloor \frac{x}{s_x} \right\rfloor + z_x, q_{\min}, q_{\max} \right). \quad (7)$$

The purpose of quantization is not merely model compression. More importantly, it aligns arithmetic precision with the resident-weight data path, accumulation staging, and mode-specific head reuse. Search and Track therefore share one calibration contract at the backbone interface so that both modalities can traverse the same transformer substrate without reformatting, while the output heads keep independent scale

TABLE II
APPROXIMATION POLICY FOR NON-LINEAR AND RE-SCALE OPERATORS

Operator	Hardware realization	Design rationale
LayerNorm denominator	Split-range reciprocal-root table	High slope near zero requires denser codebook allocation.
Softmax exponent	Max-subtracted exponent table	Stabilizes dynamic range before score normalization.
Softmax normalization	Reciprocal table with trimmed domain	Avoids divider hardware while preserving ranking behavior.
GELU + re-scale	Composed curve table	Collapses two serial non-linear stages into one access.
Standalone quantization	re-Shift-address re-scale table	Removes general multiplier from address generation path.

tables because their activation statistics differ across Search, Event, Track, and Mask prediction.

A second design choice is *shift-aligned quantization*. Instead of allowing arbitrary scale factors everywhere, HBTXR prefers scales that can be implemented through short integer shifts or small constant multipliers at the point where addresses or rescale factors are generated. This reduces controller complexity and makes quantized activation routing easier to integrate with table-driven non-linear blocks.

2) *Non-linear Approximation*: The hardware-unfriendly operators in HBTXR are LayerNorm, Softmax, GELU, and output re-scaling. Following the deployment philosophy of HG-PIPE [11], but using HBTXR-specific implementation terms, these operators are mapped to *table-driven curve blocks*. The key idea is to replace expensive arithmetic with compact codebooks indexed by quantized activation statistics.

HBTXR uses three approximation rules. First, *shift-address table indexing* generates look-up addresses with subtraction and bit shifting instead of a general multiplier. Second, *curve composition* fuses consecutive transfer functions when their boundaries are fixed in the deployed graph, such as GELU followed by re-scaling. Third, *split-range coding* allocates finer address density to regions where the function slope is steep, such as reciprocal and reciprocal-root near zero, and coarser density to the tail. A final calibration step, called *data-aware table trimming*, narrows the table domain to the activation range observed after quantization so that codebook entries are not wasted on clipped extremes.

The approximation policy in Table II is intentionally conservative: it preserves the ordering and monotonicity properties that are important for attention and head outputs while shifting cost from DSP-heavy arithmetic to LUTs, short BRAM tables, and lightweight address logic.

C. Accelerator Architecture

1) *Intra-Stage Dataflow*: The backbone datapath is decomposed into two arithmetic families. The first is the *stored-weight matrix engine*, which executes operators whose coefficient matrices are stationary during inference. Typical examples are patch embedding, QKV projection, output projection, and the two feed-forward layers. The second is the *context-mixing matrix engine*, which executes operators whose effective weights are generated online from runtime tensors, most notably score computation and value mixing in self-attention.

Formally, the stored-weight path evaluates

$$\mathbf{Y} = \mathbf{X}\mathbf{W}, \quad (8)$$

where $\mathbf{W}$ is a learned projection kept resident in on-chip memory banks. The context-mixing path evaluates

$$\mathbf{R} = \text{softmax}(\mathbf{Q}\mathbf{K}^\top)\mathbf{V}, \quad (9)$$

where the effective mixing coefficients are produced online and cannot be treated as stationary weights. The first path is naturally compatible with output-stationary accumulation. The second path requires a context reservoir because the consumption order of $\mathbf{V}$ differs from the generation order of the score map.

In HBTXR, the stored-weight engine uses line-oriented token stripes and banked resident weights. The context-mixing engine uses a score-ready release rule: tokens enter the reservoir continuously, but value mixing is issued only after the corresponding score set has been normalized. This allows the accelerator to preserve local streaming where possible while respecting the global dependency barrier imposed by attention.

2) *Multi-Stage Pipeline*: At the cross-stage level, HBTXR does not rely on one uniform pipeline style. Instead, it adopts a *dual-scale overlap chain*. *Set-synchronous passes* handle operations that require whole-token or whole-branch readiness, such as score normalization, branch merge, and full Search refresh. *Fragment-stream passes* handle operations that admit immediate release once local dependencies are satisfied, such as patch projection, feed-forward processing, and lightweight head decoding.

Let W_i denote the workload of stage i , p_i the parallel factor assigned to that stage, and δ_i the local control overhead. We define the stage cadence as

$$C_i = \left\lceil \frac{W_i}{p_i} \right\rceil + \delta_i, \quad (10)$$

and the system cadence as

$$C_{\text{sys}} = \max_i C_i. \quad (11)$$

The design objective is therefore not to minimize one kernel latency in isolation, but to compress the spread of C_i across all stages. HBTXR accomplishes this with *stage-cadence equalization*: more parallel resources are allocated to stages that would otherwise dominate C_{sys} , while memory-bank layout is adjusted so that resident weights fit naturally into FPGA BRAM widths instead of leaving partially used banks.

This strategy also motivates *stage-local control*. Rather than using one centralized controller to orchestrate all blocks, HBTXR gives each stage a small local FSM and connects neighboring stages with ready/valid handshakes through elastic lanes. This keeps routing localized, absorbs short-term cadence mismatch, and reduces the risk of deadlock when Search and Track activate different sets of heads.

3) *Streaming-Oriented Feature Delivery*: Memory movement is often more expensive than arithmetic in embedded XR deployment. HBTXR therefore distinguishes between *refresh delivery* and *resident delivery*. Search mode uses a reseed path that stages the incoming frame tokens, event summary tokens,

Figure placeholder
Final artwork pending

Fig. 3. HBTXR hardware overview using the terminology of this paper. The accelerator is organized as a dual-scale overlap chain with set-synchronous passes, fragment-stream passes, elastic lanes, context reservoirs, and a resident feature ring shared across Search and Track.

and temporary synchronization context into scratch storage, regenerates a reliable anchor, and writes the accepted anchor into the resident feature ring. Track mode uses resident delivery: only event tokens, the prior state token, and the small amount of context needed by the Track head are circulated.

This organization can be summarized by the on-chip memory model

$$M_{\text{chip}} = M_{\text{resident}} + M_{\text{elastic}} + M_{\text{scratch}}, \quad (12)$$

where M_{resident} stores persistent Search anchors and previous-state context, M_{elastic} corresponds to the inter-stage lanes, and M_{scratch} is mode-dependent temporary storage. Search has the larger M_{scratch} footprint because it rebuilds global anchors. Track keeps M_{scratch} small and instead depends on the resident feature ring.

D. Patch Embedding Module

The patch embedding module is the modality adaptation front end of HBTXR. It receives either a canonicalized frame patch or an accumulated event tensor and converts it into a common token width for the shared backbone. Because Search and Track expose different statistics, the front end keeps separate input adapters even though both ultimately emit tokens compatible with the same backbone interface.

For frame inputs, the front end emphasizes stable anchor recovery. It therefore performs deterministic window extraction, line-based buffering, and resident-weight patch projection before tokens are tagged for Search-mode processing. For event inputs, the front end emphasizes temporal responsiveness. The event tensor is already sparse and motion-focused; therefore the event adapter prioritizes low-overhead accumulation output, token packing, and previous-state injection so that the backbone receives a compact representation of recent motion together with the last valid pupil hypothesis.

The front-end mapping is summarized as

$$\mathbf{T}_t^f = \Pi_f(I_t), \quad \mathbf{T}_t^e = \Pi_e(V_t, \mathbf{s}_{t-1}), \quad (13)$$

where $\Pi_f(\cdot)$ and $\Pi_e(\cdot)$ denote the frame and event tokenizers, respectively. By keeping the output width of both tokenizers

identical, HBTXR allows one shared backbone engine to process both Search and Track inputs without interface conversion overhead.

E. Transformer Module

The transformer module forms the dominant compute core of HBTXR. Each block contains normalization, QKV generation, score formation, score normalization, value mixing, output projection, residual addition, and feed-forward processing. The stored-weight matrix engine serves the projection and MLP sublayers, whereas the context-mixing matrix engine serves the score and value interactions. This decomposition reflects the actual hardware access pattern more faithfully than treating the entire block as one monolithic operator.

A second design choice is *resident-weight deployment*. The shared backbone weights are kept on chip as aggressively as the target device allows, because re-fetching the dominant transformer parameters would erase the latency advantage of Track mode. Search and Track therefore reuse the same resident projection banks and differ mainly in the input tokens issued by the front end and in the output heads activated afterward. In this sense, the accelerator is shared in arithmetic but mode-aware in data delivery.

The transformer module also uses *bank-fit factorization*. When a projected weight tile would leave partially filled BRAM banks, the parallel factor is adjusted so that the bank width is used more efficiently even if the raw MAC count of the stage stays constant. This detail is directly inspired by the practical lesson of HG-PIPE [11]: end-to-end efficiency depends on memory-bank fit and cadence equalization at least as much as on algorithmic FLOP count.

F. Head Module

The head module contains the Search, Event, Track, Mask, and auxiliary gaze heads. Although these heads are much lighter than the shared backbone, they are critical because they encode the functional asymmetry between Search and Track. Search activates the Search head, objectness-related logic, and the Mask head to reacquire a reliable anchor and validate the eye region. Track activates the Track head and event-conditioned

Figure placeholder
Final artwork pending

Fig. 4. Execution timeline of the proposed dual-scale overlap chain. Set-synchronous passes appear at attention barriers and Search refresh points, whereas fragment-stream passes dominate patch projection, feed-forward processing, and Track-head decoding.

TABLE III
MODE-DEPENDENT HARDWARE MAPPING POLICY OF HBTXR

Mode	Active front end	Dominant backbone behavior	Resident data usage	Active heads	Controller emphasis
Search	Frame patchizer + event summary adapter	Set-synchronous refresh with full branch coordination	Update reseed bank and commit a new anchor to the resident feature ring	Search, Mask, objectness, gaze refinement	Anchor reacquisition, validity reset, confidence check
Track	Event patchizer + previous-state injector	Fragment-stream update over resident anchor context	Reuse resident feature ring and state register file	Track, Event, gaze refinement	Low-latency response, minimum control churn
Scheduled hybrid	Dynamic switch between both paths	Alternates refresh and resident passes according to quality signals	Commit-on-success update policy	Mode-dependent head activation	Balance robustness and update rate

quality logic to update the prior state with minimal delay. The gaze output is generated from the final accepted state rather than from an entirely separate heavyweight branch.

Hardware-wise, the heads are implemented as fragment-stream decoders with mode-dependent gating. Only the heads required by the current mode are released by the controller, and the shared feature reader broadcasts the final token bundle to the selected head set. This avoids the wasteful behavior of evaluating all heads during every invocation while keeping the head latency nearly hidden behind backbone output serialization.

G. CPU–FPGA Workload Partition (Scheduler)

HBTXR adopts a heterogeneous CPU–FPGA organization because the runtime semantics of Search and Track are control-heavy at the system boundary but tensor-heavy inside the network core. The CPU is responsible for frame and event acquisition, accumulation-window management, closed-eye handling, similarity and confidence evaluation, and the final Search/Track decision. The FPGA is responsible for tokenization, shared-backbone inference, active-head execution, and resident-context update.

At runtime, the CPU sends a mode packet containing the current operation mode, the previous accepted state, and scheduler-side quality metadata. The FPGA-side controller interprets this packet and releases the corresponding front end, backbone path, and head combination. Because Search and Track are not simply two different inputs to one static graph but two distinct operating states of the deployed system, the

controller is not an incidental implementation detail; it is part of the algorithm–hardware contract.

The end-to-end latency can be decomposed as

$$T_{e2e} = T_{\text{host}} + T_{\text{io}} + T_{\text{front}} + T_{\text{backbone}} + T_{\text{head}} + T_{\text{sync}}, \quad (14)$$

where the terms denote host scheduling, host-device transfer, front-end tokenization, shared-backbone execution, head decoding, and synchronization overhead, respectively. Let α denote the fraction of Search invocations. Then the average runtime is

$$T_{\text{avg}} = \alpha T_{\text{Search}} + (1 - \alpha) T_{\text{Track}}. \quad (15)$$

This expression highlights the purpose of the hardware design: HBTXR is not optimized for one uniform synthetic throughput number, but for the actual Search/Track operating distribution produced by the scheduler.

V. EXPERIMENTAL RESULTS

A. Experiments Setup

Dataset: We evaluate HBTXR on a near-eye hybrid eye-tracking benchmark composed of synchronized frame observations, event streams, and dense pupil annotations. Each sample contains a canonicalized frame patch, a polarity-split event tensor, the previous tracking state, and geometry-aware supervision targets. The dataset is divided in a subject-disjoint manner for training, validation, and testing so that cross-subject generalization is reflected in the final report.

TABLE IV
EXPERIMENTAL SETUP SUMMARY

Item	Setting
Training protocol	Two-stage Search pretrain + hybrid fine-tune
Input frame size	256×256
Input event size	$2 \times 256 \times 256$
Event policy	Fixed-count
Default event count	5000
Accumulation	Fast causal accumulation
Resize policy	Direct square canonical resize
Scheduler signals	Search conf., track conf., similarity, density, eye state
Optimizer	AdamW
Batch size	8
Learning rate	3×10^{-4}
Weight decay	1×10^{-4}
Runtime platform	Host-device heterogeneous execution

Software / Hardware: The algorithm is trained in PyTorch and exported to a host-device runtime that mirrors the proposed CPU-FPGA deployment model. The host handles sensor I/O and scheduling, while the accelerator-side execution model assumes modality adaptation, shared-backbone inference, and head decoding on FPGA. The current JETCAS draft contains verified latency numbers for the deployed system organization, but finalized post-place-and-route resource utilization and board-level power remain to be inserted from the final implementation report.

Hyper-parameters: The main configuration is summarized in Table IV. HBTXR uses two-stage training, 256×256 canonicalized inputs, fixed-count event windows with 5000 events, fast causal accumulation, AdamW optimizer, batch size of 8, learning rate of 3×10^{-4} , and weight decay of 1×10^{-4} . Search pretraining is followed by hybrid fine-tuning with Search, Event, Track, Mask, and auxiliary supervision terms.

Workstation: Training and validation are conducted in a standard GPU-based workstation environment, while runtime evaluation follows the host-device execution model described in Section V. Since the current source draft does not provide a finalized workstation sentence with exact CPU/GPU specifications, this line is intentionally kept generic and should be replaced by the verified configuration before submission.

Runtime Instrumentation: In addition to pupil and gaze accuracy, we monitor Search success rate, end-to-end latency, and effective update rate. The deployment analysis further separates host scheduling, host-device transfer, front-end adaptation, backbone execution, head decoding, and synchronization as conceptual latency terms, even though only the final end-to-end mode latencies are currently verified numerically in the source draft.

B. Proposed HBTXR Evaluation

1) *Event-centric vs. Frame-centric vs. Hybrid Accuracy and Latency:* Table V reports the operational comparison between the frame-centric Search path, the event-centric Track path, and the scheduled hybrid system. Search mode achieves 0.58 px pupil error, 0.91° gaze error, 97.8% Search success rate, and

TABLE V
OPERATIONAL COMPARISON OF FRAME-CENTRIC, EVENT-CENTRIC, AND HYBRID MODES

Mode	Pupil Err. (px)	Gaze Err. (deg)	Latency (ms)	Rate (Hz)
Frame-driven Search	0.58	0.91	0.93	1075
Event-driven Track	0.42	0.68	0.57	1754
Hybrid scheduled	0.45	0.71	0.63	1587

TABLE VI
HARDWARE IMPLEMENTATION RESULTS OF HBTXR

Metric	Search	Track	Scheduled	Status
End-to-end latency (ms)	0.93	0.57	0.63	verified in draft
Effective update rate (Hz)	1075	1754	1587	verified in draft
Search success rate (%)	97.8	–	97.8	verified in draft
LUT / DSP / BRAM	TBD	shared design	TBD	post-P&R report
Board power	TBD	TBD	TBD	board measurement pending
Latency breakdown	host / io / front / backbone / head / sync	host / io / front / backbone / head / sync	averaged	profiler table pending

0.93 ms latency. Track mode achieves 0.42 px pupil error, 0.68° gaze error, and 0.57 ms latency, corresponding to 1754 Hz effective update rate. Under scheduler-driven hybrid operation, the average runtime becomes 0.63 ms while preserving strong accuracy. This result is consistent with the intended operating policy of HBTXR: Search is a refresh path invoked only when necessary, whereas Track dominates steady-state low-latency update.

The Track row should be interpreted as an event-centric state-conditioned update rather than as a fully stand-alone detector without prior context. Nevertheless, the comparison still exposes the system-level asymmetry that motivates the hardware architecture: Search benefits from stronger global refresh, while Track benefits from resident-context reuse and reduced control churn.

2) *Hardware Implementation Results (Resources, Power, Latency Breakdown):* Table VI summarizes the current state of the hardware evaluation. The mode-specific end-to-end latency and effective update rate are retained from the verified draft results. However, finalized LUT/DSP/BRAM numbers and measured board power are intentionally left as TBD because the source draft does not yet include post-place-and-route or board-level measurement tables. This keeps the manuscript structurally complete without fabricating unsupported values.

Even with these pending entries, the verified mode latency gap is already informative. The reduction from 0.93 ms in Search to 0.57 ms in Track supports the rationale of the resident feature ring, mode-dependent head gating, and the dual-scale overlap chain. In other words, the hardware contribution of HBTXR is not just raw arithmetic acceleration but a deployment path that amplifies the algorithmic distinction between refresh-oriented Search and state-persistent Track.

Figure placeholder
Final artwork pending

Fig. 5. Evaluation panels for the revised hardware narrative. Recommended final artwork can combine qualitative Search/Track outputs, mode transitions from the scheduler, and the latency decomposition into host, transfer, front-end, backbone, head, and synchronization components.

TABLE VII
COMPARISON WITH REPRESENTATIVE EYE TRACKING ALGORITHMS USING SOURCE-REPORTED METRICS

Work	Mod.	Primary Task	Reported Accuracy	Reported Runtime	Reloc.	Note
Etracker	Frame	Gaze	0.53° gaze accuracy	30–60 Hz	Yes	Mobile near-eye tracker
HE-Tracker	Frame	Gaze	3.655° gaze error	48 fps on AR HMD	Yes	Head-eye coordination model
E-Track	Event	Pupil	3.68 px median error	66.4 ms U-Net inference	No	Event-only detector
FACET	Event	Pupil	0.2030 px PE	0.5302 ms inference	No	Ellipse-based detector
EX-Gaze	Hybrid	Pupil / tracking	1.33 px (saccade), 1.49 px (smooth pursuit)	0.39–0.43 s per 1 s stream on Orin Nano	Yes	2 KHz hybrid system
HBTXR	Hybrid	Pupil + Gaze	0.42 px pupil err.; 0.68° gaze err.	0.57 ms Track mode	Yes	Unified Search/Track system

C. Comparison with Other Eye Tracking Algorithm

Table VII summarizes representative prior eye-tracking systems using the metrics reported in their source papers. Because the referenced works differ in task definition, dataset, runtime platform, and evaluation protocol, the comparison is intentionally source-native rather than forcibly normalized. Even under this conservative presentation, HBTXR stands out by combining sub-millisecond Track latency with both pupil and gaze metrics under one Search/Track framework.

D. Comparison with Other Accelerator

Table VIII compares HBTXR with representative accelerator-oriented systems across three categories: eye tracking accelerators, transformer accelerators, and object detection / tracking accelerators. EyeCoD represents frame-oriented eye-tracking acceleration, SEE-D and JaneEye represent event-centric FPGA/ASIC designs, HG-PIPE is included as a representative transformer accelerator with stage-overlap and LUT-oriented operator reformulation, and AHCO-YOLO is included as the reference style target for algorithm–hardware co-optimization in detection systems. This comparison clarifies that HBTXR is closest in spirit to AHCO-style co-design while borrowing selected transformer-acceleration lessons from HG-PIPE.

Compared with HG-PIPE, the objective of HBTXR is different: HG-PIPE is optimized for high-throughput ViT inference under uniform batch semantics, whereas HBTXR optimizes a mode-switching Search/Track workload with strong system-level asymmetry. Compared with eye-tracking accelerators such as EyeCoD, SEE, and JaneEye, HBTXR places more

emphasis on the shared transformer backbone and scheduler-visible operating modes. These distinctions justify the revised terminology adopted in Section V.

E. Ablation Study

Table IX summarizes the ablation study. Removing geometry-stable supervision degrades pupil error from 0.42 to 0.49 pixels and gaze error from 0.68° to 0.80°. Removing decoupled heads increases both error and latency. Disabling the Search/Track scheduler leads to the largest latency increase, from 0.57 ms to 0.71 ms, showing that runtime adaptivity is central to the system value. Removing resident-feature circulation leaves accuracy unchanged but increases latency to 0.69 ms, showing that the main benefit of the revised hardware mapping is deployment efficiency rather than statistical accuracy.

In addition to the numerical ablation, the revised hardware description suggests two qualitative deployment lessons. First, collapsing the dual-scale overlap chain into a uniform execution style would reintroduce idle slots at attention barriers and weaken the latency advantage of Track mode. Second, disabling head-select gating would activate Search-only logic during Track, increasing unnecessary work without improving accuracy. These observations support the use of HBTXR-specific hardware terminology because the critical unit of optimization is not a generic transformer block, but a mode-conditioned Search/Track system.

VI. CONCLUSION

This paper presented HBTXR, an algorithm–hardware co-designed hybrid eye-tracking framework for on-device XR. The

TABLE VIII
COMPARISON WITH REPRESENTATIVE ACCELERATORS USING SOURCE-REPORTED METRICS

Category	System	Platform	Reported Throughput / Latency	Reported Efficiency	Positioning w.r.t. HBTXR
Eye tracking	EyeCoD	ASIC-style frame accelerator	240 FPS target throughput	154.32 mW silicon power at 370 MHz	Frame-oriented predict-focus pipeline
Eye tracking	SEE-D	FPGA SoC	0.70 ms latency	2.29 mJ / inference	Event-centric sparse pipeline
Eye tracking	JaneEye	12-nm ASIC	0.5 ms; 2000 FPS	18.9 μ J / frame	Event-centric ASIC co-design
ViT	HG-PIPE	VCK190 FPGA	7118 images / s	381.0 GOPS / W	Stage-overlap transformer accelerator reference
Obj. det. / tracking	AHCO-YOLO-T	ZCU104 FPGA	79.8 FPS	41.9 FPS / W; 80.5 GOPS / W	AHCO-style algorithm-hardware co-optimization reference
Hybrid eye tracking	HBTXR	CPU-FPGA host-device	0.57 ms Track latency; 1754 Hz	TBD	Runtime-adaptive Search/Track co-design

TABLE IX
ABLATION STUDY OF HBTXR

Variant	Pupil Err. (px)	Gaze Err. (deg)	Latency (ms)
Full HBTXR	0.42	0.68	0.57
w/o geometry-stable supervision	0.49	0.80	0.58
w/o decoupled heads	0.47	0.75	0.62
w/o Search/Track scheduler	0.46	0.73	0.71
w/o resident-feature circulation	0.42	0.68	0.69

main idea is to move beyond undifferentiated multimodal regression and instead treat hybrid eye tracking as a Search/Track system with explicit runtime semantics. Search provides robust frame-guided relocalization, Track provides low-latency event-guided residual update, and the scheduler couples both with deployment-aware hardware execution.

By reorganizing HBTXR in an AHCO-style narrative, this manuscript highlights that the main contribution is not a single model block or a single accelerator trick, but the joint treatment of supervision, transformer design, runtime control, and hardware mapping. The revised hardware description further grounds the design in HG-PIPE-inspired stage-overlap ideas while introducing HBTXR-specific terminology and mode semantics such as set-synchronous passes, fragment-stream passes, context reservoirs, and resident-feature circulation. Experimental results show that HBTXR achieves 0.42 px pupil error, 0.68° gaze error, and 0.57 ms Track latency while maintaining robust Search recovery. Future work should finalize board-level resource and power measurements, strengthen the quantitative hardware table with post-layout numbers, and further refine the quantization and slimming flow for full implementation closure.

REFERENCES

- [1] A. Plopski, T. Hirzle, N. Norouzi, L. Qian, G. Bruder, and T. Langlotz, "The eye in extended reality: A survey on gaze interaction and eye tracking in head-worn extended reality," *ACM Computing Surveys*, vol. 55, no. 3, pp. 1–39, 2022.
- [2] H. Kwon, K. Nair, J. Seo, J. Yik, D. Mohapatra, D. Zhan, J. Song, P. Capak, P. Zhang, and P. Vajda *et al.*, "Xrbench: An extended reality machine learning benchmark suite for the metaverse," *Proceedings of Machine Learning and Systems*, vol. 5, pp. 1–20, 2023.
- [3] L. Chen, Y. Li, X. Bai, X. Wang, Y. Hu, M. Song, L. Xie, Y. Yan, and E. Yin, "Real-time gaze tracking with head-eye coordination for head-mounted displays," in *Proc. IEEE ISMAR*, 2022, pp. 82–91.
- [4] N. Li, A. Bhat, and A. Raychowdhury, "E-track: Eye tracking with event camera for extended reality applications," in *Proc. IEEE AICAS*, 2023, pp. 1–5.
- [5] G. Zhao, Y. Yang, J. Liu, N. Chen, Y. Shen, H. Wen, and G. Lan, "EV-Eye: Rethinking high-frequency eye tracking through the lenses of event cameras," *Advances in Neural Information Processing Systems*, vol. 36, pp. 62169–62182, 2023.
- [6] J. Ding, Z. Wang, C. Gao, M. Liu, and Q. Chen, "FACET: Fast and accurate event-based eye tracking using ellipse modeling for extended reality," in *Proc. IEEE ICRA*, 2025.
- [7] N. Chen, Y. Shen, T. Zhang, Y. Yang, and H. Wen, "EX-Gaze: High-frequency and low-latency gaze tracking with hybrid event-frame cameras for on-device extended reality," *IEEE Transactions on Visualization and Computer Graphics*, 2025.
- [8] H. You, C. Wan, Y. Zhao, Z. Yu, Y. Fu, J. Yuan, S. Wu, S. Zhang, Y. Zhang, C. Li *et al.*, "EyeCoD: Eye tracking system acceleration via FlatCam-based algorithm and accelerator co-design," in *Proc. ISCA*, 2022.
- [9] B. Zhang, Y. Gao, J. Li, and H. K.-H. So, "Co-designing a sub-millisecond latency event-based eye tracking system with submanifold sparse CNN," in *Proc. CVPR Workshops*, 2024.
- [10] T. Han, A. Li, Q. Chen, and C. Gao, "JaneEye: A 12-nm 2K-FPS 18.9- μ J/frame event-based eye tracking accelerator," in *Proc. ASP-DAC*, 2026.
- [11] Q. Guo, J. Wan, S. Xu, M. Li, and Y. Wang, "HG-PIPE: Vision transformer acceleration with hybrid-grained pipeline," 2024.
- [12] J. Kim, J.-K. Kang, and Y. Kim, "AHCO-YOLO: An algorithm-hardware co-optimization framework for energy-efficient and real-time object detection on edge devices," *IEEE Trans. VLSI Syst.*, 2025.
- [13] C. Lin, J. Hou, and Z. Li, "TimeLens: Event-based video frame interpolation," *Proc. CVPR*, 2021.
- [14] R. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. Berg, W.-Y. Lo, *et al.*, "Segment anything," *Proc. ICCV*, 2023.
- [15] A. Kirillov *et al.*, "Grounded-SAM: Marrying grounding DINO with segment anything," 2023.